eventHandlerManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"eventHandlerManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `eventHandlerManagement` service of the EventHandler system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query-subscriptions . . . 4
2.2 operation update-subscription . . . 4
2.3 operation remove-subscription . . . 5

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct SubscriptionListResponse . . . 6
3.3 struct SubscriptionRequest . . . 6
3.4 struct SubscriptionResponse . . . 6
3.5 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `eventHandlerManagement` service interface of the EventHandler system, which provides operator management of all event subscriptions.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `eventHandlerManagement` service interface

This document provides the Interface Design Description IDD to the *eventHandlerManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `eventHandlerManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `eventhandler`, port 8455 and base path `/eventhandler/management`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query-subscriptions`	GET	`/subscriptions`
`update-subscription`	PUT	`/subscriptions/{subscriptionId}`
`remove-subscription`	DELETE	`/subscriptions/{subscriptionId}`

2 Service Operations

The interfaces of the `eventHandlerManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `eventHandlerManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 GET /eventhandler/management/subscriptions?page=*{page}*&size=*{size}*&direction=*{direction}*&sortField=*{sortField}*

Operation: query-subscriptions
Output: SubscriptionListResponse

Returns a page of all subscriptions.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sortField* (String, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /eventhandler/management/subscriptions?page=<page>&size=<size>&direction=<direction>&sortField=<
    sortField> HTTP/1.1
Host: eventhandler:8455
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An query-subscriptions invocation.

```
{
  "entries": [
    {
      "id": 42,
      "subscriberSystemName": "string",
      "eventType": "temperatureAlarm",
      "notifyUri": "https://display:8080/events",
      "matchMetadata": true
    }
  ],
  "count": 1
}
```

Listing 2: An query-subscriptions response.

2.2 PUT /eventhandler/management/subscriptions/*{subscriptionId}*

Operation: update-subscription
Input: SubscriptionRequest
Output: SubscriptionResponse

Replaces a subscription.

Path parameter: *subscriptionId*.

A successful call returns `200 OK` with the output in the body.

```
PUT /eventhandler/management/subscriptions/<subscriptionId> HTTP/1.1
Host: eventhandler:8455
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "eventType": "temperatureAlarm",
  "notifyUri": "https://display:8080/events",
  "matchMetadata": true
}
```

Listing 3: An update-subscription invocation.

```
{
  "id": 42,
  "subscriberSystemName": "string",
  "eventType": "temperatureAlarm",
  "notifyUri": "https://display:8080/events",
  "matchMetadata": true
}
```

Listing 4: An update-subscription response.

2.3 DELETE /eventhandler/management/subscriptions/*{subscriptionId}*

Operation: remove-subscription
Input: String

Removes a subscription.

Path parameter: *subscriptionId*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /eventhandler/management/subscriptions/<subscriptionId> HTTP/1.1
Host: eventhandler:8455
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An remove-subscription invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.5, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct SubscriptionListResponse

List of subscriptions.

Object Field	Value Type	Description
"entries"	Array<SubscriptionResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.3 struct SubscriptionRequest

A subscription to an event type.

Object Field	Value Type	Description
"eventType"	String	Event type, camelCase.
"notifyUri"	String	HTTPS endpoint to which matching events are posted.
"matchMetadata"	Boolean	Whether only events whose metadata contain `metadataFilter` are delivered.
"metadataFilter"	Object<String>	Metadata an event must contain to be delivered.
"startsAt"	Object	Start of the subscription's validity window.
"expiresAt"	Object	End of the validity window.

3.4 struct SubscriptionResponse

A stored subscription.

Object Field	Value Type	Description
"id"	Number	Identifier of the subscription.
"subscriberSystem Name"	String	System that owns the subscription.
"eventType"	String	Event type, camelCase.
"notifyUri"	String	HTTPS endpoint to which matching events are posted.
"matchMetadata"	Boolean	Whether only events whose metadata contain `metadataFilter` are delivered.
"metadataFilter"	Object<String>	Metadata an event must contain to be delivered.
"startsAt"	Object	Start of the subscription's validity window.
"expiresAt"	Object	End of the validity window.
"createdAt"	Object	UTC time of creation.
"updatedAt"	Object	UTC time of the last change.

3.5 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “eventHandlerManagement – Service Description (EventHandler),” Eclipse Arrowhead, SD 5.2.2, 2026, eventHandler-v5.2.2-eventHandlerManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	